

TEXT CLASSIFICATION ON IMDB MOVIE REVIEWS

Objective

The objective of this project is to build a text classification model that can classify movie reviews as either positive or negative using the IMDb Movie Reviews dataset. This involves data preprocessing, feature extraction, model training, and evaluation.

Dataset

The IMDb Movie Reviews dataset contains 100,000 movie reviews labeled as positive or negative. We will use this dataset to train and evaluate different classification models.

Process overview

1. Data loading

Rationale: This step involves loading the IMDb movie review data, which is split into separate directories for positive and negative reviews. We load and combine them into a single DataFrame for further processing.

```
# Load IMDb Movie Reviews dataset
url = 'https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz'
!wget {url} -O imdb_reviews.tar.gz
!tar -xf imdb_reviews.tar.gz
```

```
--2024-08-06 20:29:18-- https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz
Resolving ai.stanford.edu (ai.stanford.edu)... 171.64.68.10
Connecting to ai.stanford.edu (ai.stanford.edu)|171.64.68.10|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 84125825 (80M) [application/x-gzip]
Saving to: 'imdb_reviews.tar.gz'
```

```
imdb_reviews.tar.gz 100%[=====>] 80.23M 5.59MB/s in 20s
```

```
2024-08-06 20:29:38 (4.04 MB/s) - 'imdb_reviews.tar.gz' saved [84125825/84125825]
```

```
def load_data(path):
    data = []
    for label in ['pos', 'neg']:
        dir_name = f'{path}/{label}'
        for file in os.listdir(dir_name):
            with open(f'{dir_name}/{file}', 'r', encoding='utf-8') as f:
                text = f.read()
                data.append((text, 1 if label == 'pos' else 0))
    return pd.DataFrame(data, columns=['review', 'sentiment'])

train_data = load_data('/content/aclImdb/train')
test_data = load_data('/content/aclImdb/test')
```

```
# Combine train and test data for processing
data = pd.concat([train_data, test_data], ignore_index=True)
data
```

	review	sentiment
0	This 1991 NBC-TV movie aired six months before...	1
1	This film to me is a very good film!! <br...	1
2	Cassandra Peterson originally created Elvira a...	1
3	This is a complex documentary that shows many ...	1
4	Jean Harlow and Clark Gable were a great on sc...	1
...	...	...
49995	My review is divided into questions that you r...	0
49996	At the least some of the sitcoms that churn en...	0
49997	Huge, waddling, grimacing tree trunk menaces f...	0
49998	What is it with studios like Paramount that ha...	0
49999	Admittedly, you can put a model airplane again...	0

50000 rows × 2 columns

2. Data preprocessing

Rationale: Preprocessing is crucial to remove noise from the text data. This includes removing HTML tags and special characters, converting text to lowercase, removing stop words, and lemmatizing words to their base forms.

```
# Data Preprocessing
def preprocess_text(text):
    # Remove HTML tags
    text = re.sub(r'<.*?>', '', text)
    # Remove special characters and numbers
    text = re.sub(r'^a-zA-Z\s', '', text)
    # Convert to lowercase
    text = text.lower()
    # Tokenize
    tokens = text.split()
    # Remove stop words
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words]
    # Lemmatize
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(word) for word in tokens]
    return ' '.join(tokens)
```

```
data['cleaned_review'] = data['review'].apply(preprocess_text)
```

```
data['cleaned_review']
```

```
0      nbctv movie aired six month john goodman bigsc...
1      film good filmi german shepherd wish god like ...
2      cassandra peterson originally created elvira t...
3      complex documentary show many thing early gay ...
4      jean harlow clark gable great screen team may ...
```

3. Feature extraction

- **Bag of Words (BoW)**

Rationale: Bag of Words is a simple yet effective way to convert text data into numerical features by counting the occurrences of each word in the text.

```
# Feature Extraction
# Bag of Words
bow_vectorizer = CountVectorizer()
X_train_bow = bow_vectorizer.fit_transform(X_train)
X_test_bow = bow_vectorizer.transform(X_test)
```

- **TF-IDF**

Rationale: TF-IDF (Term Frequency-Inverse Document Frequency) is used to transform the text data into numerical features by considering the importance of words in the text relative to the corpus.

```
# TF-IDF
tfidf_vectorizer = TfidfVectorizer()
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)
```

- **BERT Embeddings**

Rationale: BERT embeddings capture the semantic meaning of words in context, providing a richer representation of text data compared to traditional methods like BoW and TF-IDF.

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased').to(device)
```

```
def get_bert_embeddings(text_list, tokenizer, model, batch_size=32):
    model.eval()
    embeddings = []

    for i in tqdm(range(0, len(text_list), batch_size)):
        batch_text = text_list[i:i + batch_size]
        inputs = tokenizer(batch_text, return_tensors='pt', padding=True, truncation=True, max_length=512).to(device)

        with torch.no_grad():
            outputs = model(*inputs)

        batch_embeddings = outputs.last_hidden_state.mean(dim=1).cpu().numpy()
        embeddings.append(batch_embeddings)

    return np.vstack(embeddings)
```

```
# Convert the data to BERT embeddings
X_train_bert = get_bert_embeddings(X_train.tolist(), tokenizer, model)
X_test_bert = get_bert_embeddings(X_test.tolist(), tokenizer, model)
```

```
100%|██████████| 1250/1250 [18:26<00:00, 1.13it/s]
100%|██████████| 313/313 [04:36<00:00, 1.13it/s]
```

4. Model training and evaluation

Evaluate the performance of each model using appropriate metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- AUC-ROC
- Visualize the confusion matrix for each model.

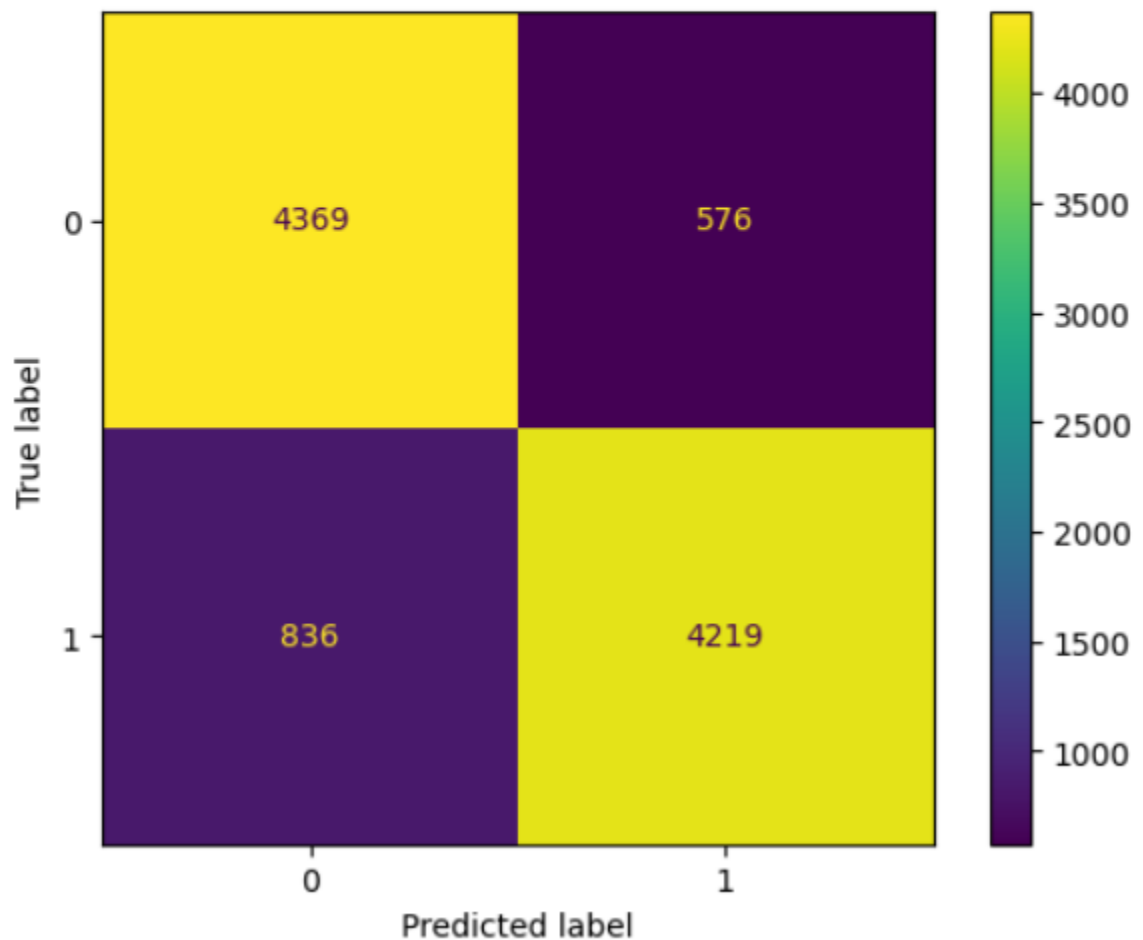
```
# Model Training
def train_evaluate_model(model, X_train, y_train, X_test, y_test):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print('Accuracy:', accuracy_score(y_test, y_pred))
    print('Precision:', precision_score(y_test, y_pred))
    print('Recall:', recall_score(y_test, y_pred))
    print('F1-Score:', f1_score(y_test, y_pred))
    print('AUC-ROC:', roc_auc_score(y_test, y_pred))
    cm = confusion_matrix(y_test, y_pred)
    ConfusionMatrixDisplay(confusion_matrix=cm).plot()
    plt.show()
```

- **Naive Bayes with BoW**

Rationale: Naive Bayes is a simple yet effective probabilistic classifier suitable for text classification tasks. It is particularly effective when using the Bag of Words feature extraction method.

```
# Naive Bayes with BoW
print("Naive Bayes with Bag of Words")
nb_bow = MultinomialNB()
train_evaluate_model(nb_bow, X_train_bow, y_train, X_test_bow, y_test)
```

Naive Bayes with Bag of Words
Accuracy: 0.8588
Precision: 0.8798748696558916
Recall: 0.8346191889218596
F1-Score: 0.8566497461928934
AUC-ROC: 0.8590689473426285



- **Linear SVM with tf-idf**

Rationale: linear svm is chosen for its efficiency in high-dimensional spaces, such as those created by tf-idf. StandardScaler is used to scale the features, improving the performance and convergence speed of the svm model.

```
# Linear SVM with TF-IDF
print("Linear SVM with TF-IDF")
linear_svm_pipeline = make_pipeline(StandardScaler(with_mean=False), LinearSVC(max_iter=1000))
train_evaluate_model(linear_svm_pipeline, X_train_tfidf, y_train, X_test_tfidf, y_test)
```

Linear SVM with TF-IDF

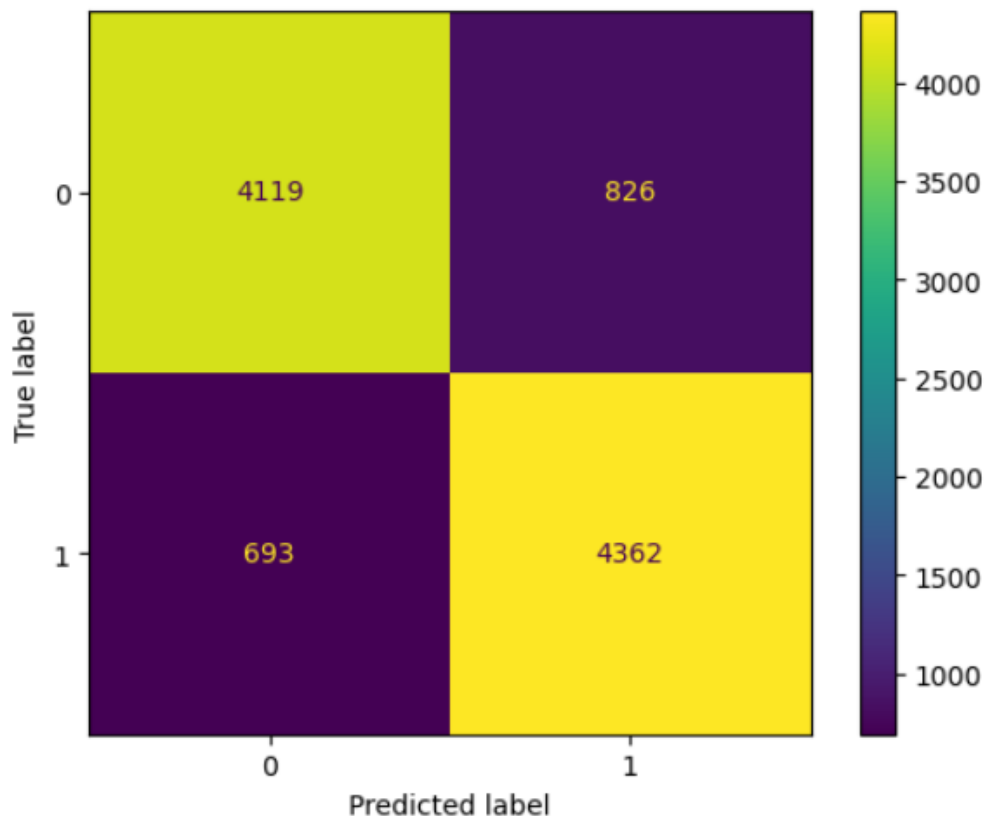
/usr/local/lib/python3.10/dist-packages/sklearn/svm/_classes.py:32: FutureWarning: The default warnings.warn(
warnings.warn(
Accuracy: 0.8481

Precision: 0.8407864302235929

Recall: 0.8629080118694362

F1-Score: 0.8517036024602167

AUC-ROC: 0.8479353001713207



- **Logistic Regression with BERT**

Rationale: Logistic Regression is a linear model used here with BERT embeddings, which provide a rich and semantically meaningful representation of the text data.

```
# Logistic Regression with BERT
print("Logistic Regression with BERT Embeddings")
lr_bert = LogisticRegression(max_iter=1000)
train_evaluate_model(lr_bert, X_train_bert, y_train, X_test_bert, y_test)
```

Logistic Regression with BERT Embeddings

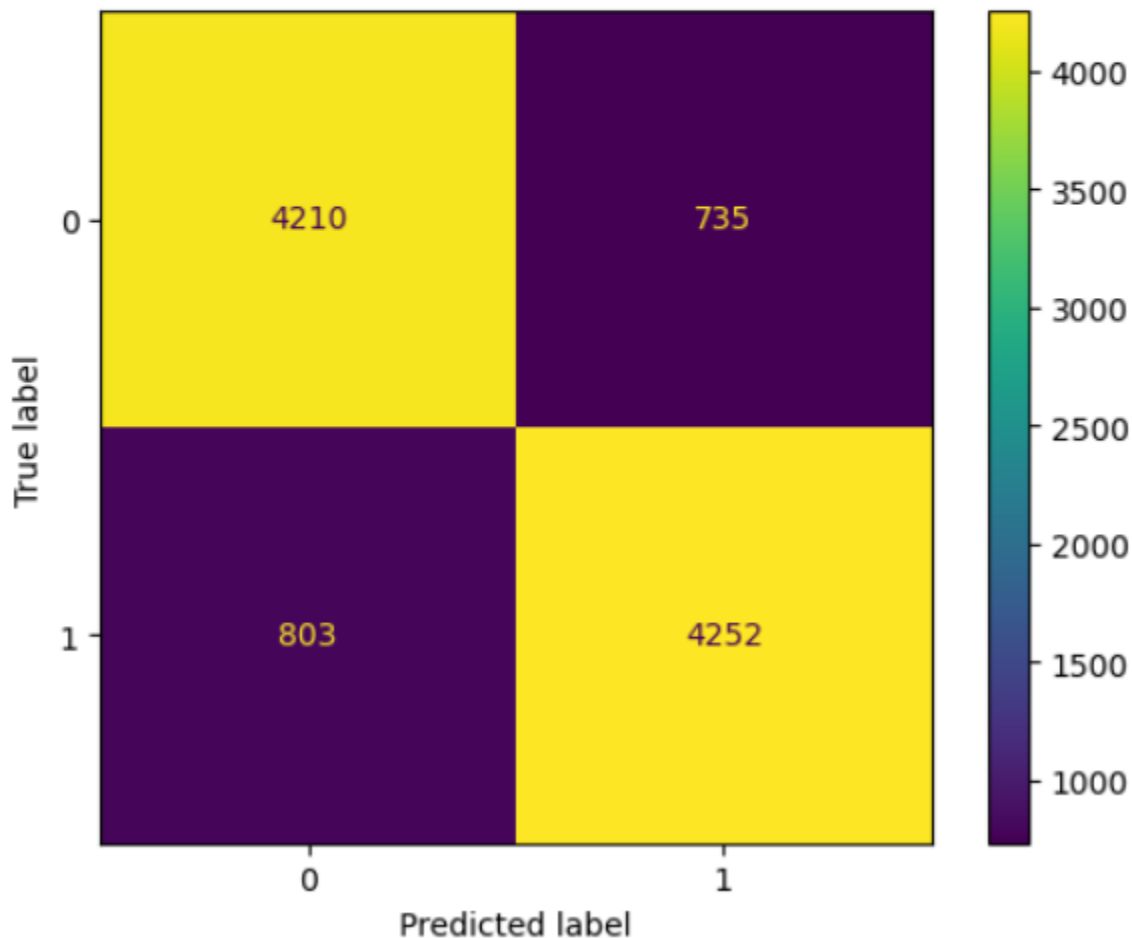
Accuracy: 0.8462

Precision: 0.8526168036895929

Recall: 0.8411473788328387

F1-Score: 0.8468432583150767

AUC-ROC: 0.846256196999837



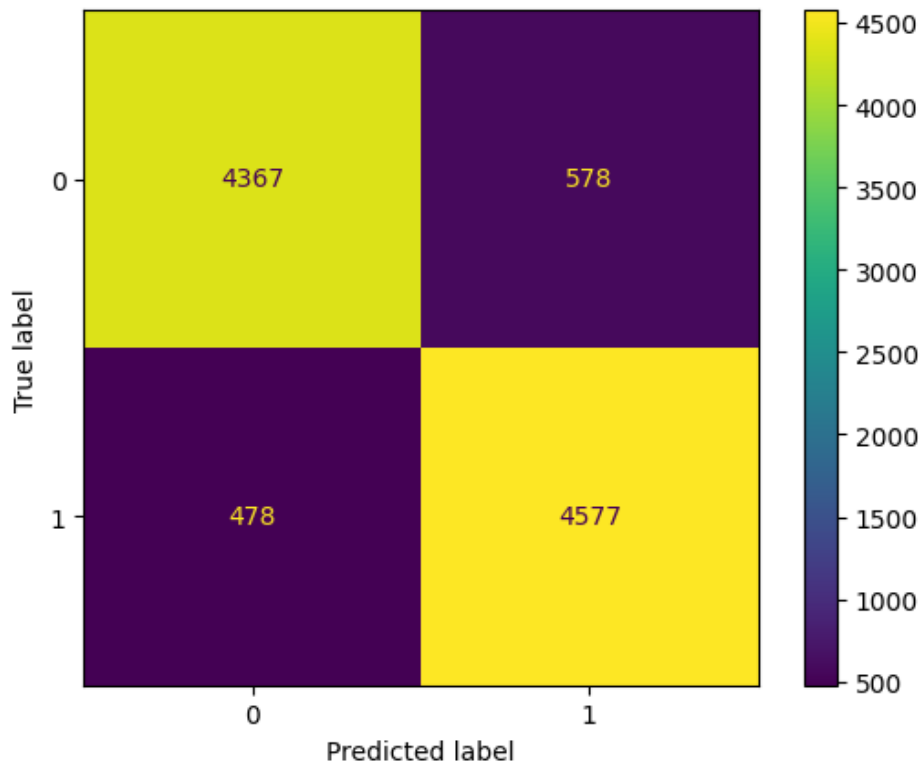
5. Handling class imbalance

Rationale: SMOTE (Synthetic Minority Over-sampling Technique) is used to handle class imbalance by generating synthetic samples for the minority class. This helps the model to better learn from both classes and improves performance metrics such as precision, recall, and F1-score.

```
# Handle Class Imbalance with SMOTE
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_tfidf, y_train)

# Logistic Regression with resampled data
print("Logistic Regression with Resampled TF-IDF Data")
lr_tfidf_resampled = LogisticRegression(max_iter=1000)
train_evaluate_model(lr_tfidf_resampled, X_train_resampled, y_train_resampled, X_test_tfidf, y_test)
```

Logistic Regression with Resampled TF-IDF Data
Accuracy: 0.8944
Precision: 0.8878758486905917
Recall: 0.9054401582591494
F1-Score: 0.8965719882468168
AUC-ROC: 0.8942772075421126



6. Hyperparameter tuning with random forest

Rationale: Grid Search is used to perform hyperparameter tuning for the Random Forest model, exploring different combinations of hyperparameters to find the best performing model.

```
# Hyperparameter Tuning with Grid Search for Random Forest
print("Random Forest with Hyperparameter Tuning")
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2],
    'bootstrap': [True, False]
}
```

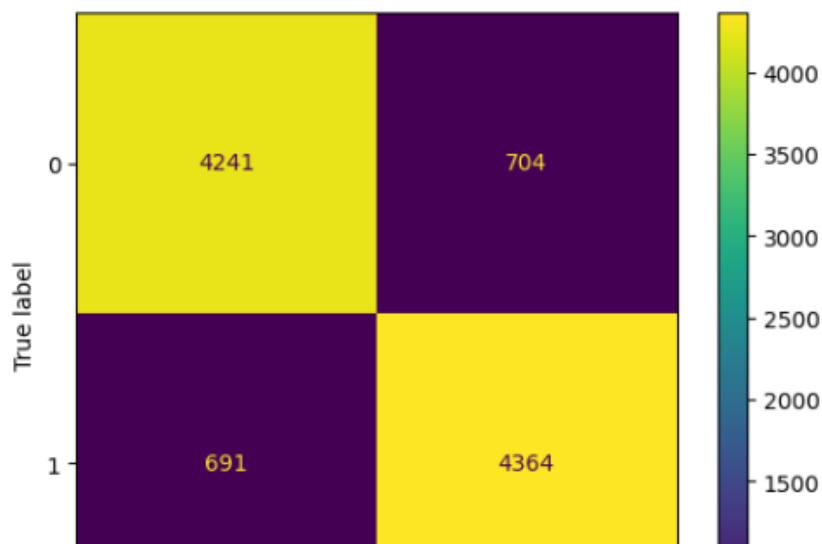
Random Forest with Hyperparameter Tuning

```
rf = RandomForestClassifier()
grid_search = GridSearchCV(estimator=rf, param_grid=param_grid, cv=3, n_jobs=-1, verbose=2, scoring='accuracy')
grid_search.fit(X_train_tfidf, y_train)
best_rf = grid_search.best_estimator_
```

Fitting 3 folds for each of 48 candidates, totalling 144 fits

```
train_evaluate_model(best_rf, X_train_tfidf, y_train, X_test_tfidf, y_test)
```

Accuracy: 0.8605
Precision: 0.8610891870560379
Recall: 0.8633036597428289
F1-Score: 0.8621950014817741
AUC-ROC: 0.860468816726824



7. Optionally, deep learning models such as LSTM.

- Set parameters for tokenization and LSTM

Rationale:

- **max_features** parameter defines the maximum number of unique words to keep in the vocabulary, focusing on the most frequent words.
- **Maxlen** is the maximum length for padding/truncating sequences to ensure a uniform input size for the LSTM.

```
# Import necessary libraries
import numpy as np
import pandas as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, Embedding, SpatialDropout1D
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split

# Parameters for tokenization and LSTM
max_features = 5000
maxlen = 300
```

- Tokenization and padding

Rationale:

- **Tokenizer**: converts text into a sequence of integers, where each integer represents a word's index in the tokenizer's word index. This prepares the text data for input to a neural network.
- **Pad_sequences**: ensures that all input sequences have the same length (maxlen), which is necessary for batch processing in LSTM layers.

```
# Tokenization and Padding
tokenizer = Tokenizer(num_words=max_features, split=' ')
tokenizer.fit_on_texts(data['cleaned_review'].values)

# Convert text data to sequences of integers
X = tokenizer.texts_to_sequences(data['cleaned_review'].values)

# Pad sequences to ensure uniform input size
X = pad_sequences(X, maxlen=maxlen)
```

- Split data into training and testing sets

Rationale: The data is split into training and testing sets, with 20% of the data used for testing. This allows for evaluation of the model's performance on unseen data. Setting `random_state` ensures reproducibility of the split.

```
# Splitting data into training and testing sets
X_train_d1, X_test_d1, y_train_d1, y_test_d1 = train_test_split(X, data['sentiment'], test_size=0.2, random_state=42)
```

- Define the LSTM Model

Rationale:

- **Sequential Model:** A simple stack of layers where each layer has exactly one input and one output.
- **Embedding Layer:** Converts integer sequences into dense vectors of fixed size (output_dim=128). It reduces the dimensionality of the input and captures semantic relationships between words.
- **SpatialDropout1D:** Applies dropout to the input of the LSTM layer to prevent overfitting by randomly setting a fraction of input units to 0 at each update during training.
- **LSTM Layer:** An LSTM layer with 100 units that captures temporal dependencies in the sequence data. Dropout and recurrent dropout help prevent overfitting by dropping out connections in both the input and recurrent connections.
- **Dense Layer:** A single output unit with a 'sigmoid' activation function, suitable for binary classification (outputting a probability between 0 and 1).

```
# Define the LSTM model
model = Sequential()
model.add(Embedding(input_dim=max_features, output_dim=128, input_length=maxlen))
model.add(SpatialDropout1D(0.2))
model.add(LSTM(100, dropout=0.2, recurrent_dropout=0.2))
model.add(Dense(1, activation='sigmoid'))
```

- Compile the Model

Rationale:

- **Loss Function:** 'binary_crossentropy' is appropriate for binary classification tasks, where the model's output is a probability.
- **Optimizer:** 'adam' is an adaptive learning rate optimization algorithm that has been shown to work well for a variety of deep learning models.
- **Metrics:** 'accuracy' is used to evaluate the model's performance during training and testing.

```
# Compile the model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

- Build the Model

Rationale: Explicitly builds the model by specifying the input shape, which helps to identify any configuration issues early on. This step ensures that the model is correctly configured before training.

```
# Build the model explicitly by passing a batch of data
model.build(input_shape=(None, maxlen))
```

- Display the model summary

Rationale: provides a summary of the model architecture, including the number of parameters for each layer, which is helpful for verifying the model configuration and understanding its complexity.

```
# Display the model summary
print(model.summary())
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	(None, 300, 128)	640,000
spatial_dropout1d_3 (SpatialDropout1D)	(None, 300, 128)	0
lstm_3 (LSTM)	(None, 100)	91,600
dense_3 (Dense)	(None, 1)	101

Total params: 731,701 (2.79 MB)

Trainable params: 731,701 (2.79 MB)

Non-trainable params: 0 (0.00 B)

None

- Train the Model

Rationale:

- **Training:** The model is trained for 5 epochs with a batch size of 64, which controls the number of samples that are propagated through the network before updating the weights.
- **Validation:** The model's performance is evaluated on the test set after each epoch, providing insights into how well it generalizes to unseen data.
- The model shows good learning capability with increasing training accuracy and decreasing training loss. The validation accuracy and loss suggest the model generalizes well, although the increase in validation loss in some epochs may indicate slight overfitting.

```
# Train the model
model.fit(X_train_dl, y_train_dl, epochs=5, batch_size=64, validation_data=(X_test_dl, y_test_dl), verbose=2)
```

Epoch 1/5

625/625 - 181s - 290ms/step - accuracy: 0.8249 - loss: 0.3904 - val_accuracy: 0.8742 - val_loss: 0.3055

Epoch 2/5

625/625 - 180s - 289ms/step - accuracy: 0.8793 - loss: 0.2985 - val_accuracy: 0.8778 - val_loss: 0.2962

Epoch 3/5

625/625 - 201s - 322ms/step - accuracy: 0.8913 - loss: 0.2717 - val_accuracy: 0.8637 - val_loss: 0.3195

Epoch 4/5

625/625 - 174s - 279ms/step - accuracy: 0.9054 - loss: 0.2372 - val_accuracy: 0.8802 - val_loss: 0.3312

Epoch 5/5

625/625 - 175s - 280ms/step - accuracy: 0.9174 - loss: 0.2111 - val_accuracy: 0.8724 - val_loss: 0.3083

<keras.src.callbacks.history.History at 0x7b3560a6aad0>

Challenges and solutions

1-High Dimensionality: TF-IDF and BERT embeddings create high-dimensional feature spaces, which can be computationally expensive. Using Linear SVM and Logistic Regression helps mitigate this issue.

2-Class Imbalance: Class imbalance can lead to biased models. We used SMOTE to balance the classes, improving model performance on both precision and recall metrics.

3-BERT Embeddings: Extracting BERT embeddings is resource-intensive. Batch processing and using a GPU help manage memory usage and improve processing time.

Conclusion

This project demonstrates the use of various text classification techniques on the IMDb Movie Reviews dataset. We explored different preprocessing steps, feature extraction methods, and model choices, providing insights into handling text data and building classification models. The use of SMOTE and hyperparameter tuning further improved the model's performance, addressing class imbalance and optimizing model parameters.